

ARC-AGI-3 Part 1: Formalization and Survey of Methods

Bo Hou · 2026-10-04 · Series: Deriving World Models for ARC-AGI-3

This is Part 1 of the series: formalize the problem, fix one notation, rewrite every existing method in that notation, and locate the gaps. Later parts derive the two-layer modeling framework from first principles.

1 Problem Formalization

Each game is an unknown deterministic machine. The agent sees only rendered frames and a little metadata; neither rules nor goals are given, so both must be inferred through interaction. The observation fields were checked against the arc-agi 0.9.9 / arcengine source.

1.1 Notation

Symbol	Meaning
$g \in \mathcal{G}$	A game; $\mathcal{G}_{pub} = 25$ public games, $\mathcal{G}_{priv} = 110$ private games
$\ell = 1, \dots, L_g$	Level index of game g ; L_g = number of levels (win_levels)
t	Real action step (global counter)
$x_t \in \{0, \dots, 15\}^{64 \times 64}$	Frame after step t (a step may return several frames; x_t is the last one)
σ_t	Game state $\in \{\text{NOT_FINISHED}, \text{WIN}, \text{GAME_OVER}\}$
c_t	Levels completed (levels_completed)
$A_t \subseteq \{0, \dots, 7\}$	Available actions: 0 = RESET, 1–4 directional, 5 interact, 6 click, 7 undo
a_t	Action; when $a_t = 6$ it carries coordinates $(u, v) \in [0, 63]^2$
$o_t = (x_t, \sigma_t, c_t, L_g, A_t)$	Full observation
z_t	True hidden state (including off-screen information such as timers or directions)
T^*, R^*	True transition and rendering
$D_t = \{(x_i, a_i, x_{i+1})\}_{i < t}$	Interaction record up to step t
$\phi : x \mapsto s$	Abstraction map from a frame to an object-level state s
$h \in \mathcal{H}$	A world-model hypothesis: T_h (transition), G_h (goal test), optional ϕ_h
Λ	Outer-layer prior (primitive library, modeling method, pretrained model)
$b_{g,\ell}$	Human baseline action count for level ℓ (baseline_actions)
$n_{g,\ell}$	Actions the AI spends on level ℓ

1.2 Dynamics

$$z_{t+1} = T^*(z_t, a_t), \quad x_t = R^*(z_t), \quad a_t \in A_t$$

- T^* is deterministic, but z_t need not be fully visible in x_t ; Tycho formalizes this as a “parameterized rendered deterministic Moore machine”.
- If the same (x, a) yields several different x' , there is off-screen state and S must include history.
- Priors: only Core Knowledge (objectness, geometry and topology, agentness; secondary sources add basic physics), no numbers, letters or other cultural symbols.

2 Scoring and Global Objective

The goal is to clear as many levels as possible, especially later ones, with as few real actions as possible; actions beyond the human baseline are penalized quadratically. Per-level score (0 if the level is not cleared):

$$r_{g,\ell} = \min(1.15, (\frac{b_{g,\ell}}{n_{g,\ell}})^2)$$

Per-game score (weighted by level index) and total score:

$$S_g = \frac{\sum_{\ell=1}^{L_g} \ell r_{g,\ell}}{\sum_{\ell=1}^{L_g} \ell}, \quad S = \frac{1}{|G|} \sum_{g \in G} S_g$$

- $b_{g,\ell}$ is the upper median of first-time human players’ action counts.
- $n_{g,\ell}$ counts every action that changes game state, exploration included; whether RESET counts is unverified.
- The 1.15 cap means an agent using about 7% fewer actions than humans already maxes out ($\sqrt{1.15} \approx 1.07$).

Global objective: a policy π (modeling plus planning) that maximizes the expected total score, i.e. minimizes per-level action counts subject to clearing levels:

$$\pi^* = \arg \max_{\pi} E_{g \sim G_{priv}} [S_g(\pi)]$$

3 Two-Layer Modeling Framework

The inner layer builds a world model on the spot inside one game and plans with it; the outer layer is a prior Λ learned offline and shared across games. Together they form a meta-learning problem.

3.1 Inner Layer: the Model–Plan Loop within One Game

At each step the hypothesis is updated from D_t , then the agent decides to explore or exploit; actions are always executed in the real environment.

$$h_t = f_{\Lambda}(D_t), \quad a_t = \pi(h_t, x_t), \quad x_{t+1} = R^*(T^*(z_t, a_t))$$

1. Observe x_t and append (x_{t-1}, a_{t-1}, x_t) to D_t .
2. Verify and revise h : T_h must agree with every record in D_t .
3. Plan: if h is trustworthy $\rightarrow$ search inside T_h for an action sequence reaching G_h ; otherwise $\rightarrow$ pick the action that best discriminates among candidate hypotheses.
4. While executing, compare T_h ’s prediction with the real x_{t+1} ; on mismatch return to step 2.

3.2 Modeling Cost

Original statement (direction level): learn rules as accurately as possible with as few probing actions as possible, i.e. $\min C_{\text{explore}} + d(T^*, T_h)$. Since T^* is unobservable, the computable version replaces d by data fit on D_t and adds a simplicity term, giving an MDL form (δ is the Kronecker delta, 1 when the two frames are identical):

$$L_{\text{fit}}(h; D_t) = \sum_{(x, a, x') \in D_t} (1 - \delta(T_h(x, a), x'))$$

$$h_t = \arg \min_{h \in H} L_{\text{len}}(h) + \lambda L_{\text{fit}}(h; D_t)$$

- L_{len} : description length of the model (shorter code generalizes better); corresponds to Rodionov's refactoring loop.
- Exact replay verification amounts to the hard constraint $L_{\text{fit}} = 0$.
- The solver is not gradient descent but discrete search over program space H : an LLM proposes edits and verifiers filter them.

3.3 Play Cost and Alternating Optimization

Planning seeks the fewest steps to G_h inside T_h . Exploration and planning alternate and serve one global objective (Section 2):

$$\min n_{g,\ell} = n_{g,\ell}^{\text{explore}} + n_{g,\ell}^{\text{play}}$$

- Exploration should be valued by how much it reduces n^{play} : learn the rules that matter for clearing levels, not as many rules as possible.
- When to pay for modeling and when to act directly is what Tycho calls active abstraction.

3.4 Outer Layer: Cross-Game Priors and Meta-Learning

General form of meta-learning: an inner algorithm f_ω adapts on a task's D^{train} and is evaluated on D^{test} ; the outer loop optimizes ω .

$$\omega^* = \arg \min_{\omega} \mathbb{E}_{T \sim p(T)} [L_T(f_\omega(D_T^{\text{train}}); D_T^{\text{test}})]$$

Here $\omega = \Lambda$, a task is a game g , D^{train} is interaction on earlier levels and D^{test} is later levels:

$$\Lambda^* = \arg \max_{\Lambda} \mathbb{E}_{g \sim G_{\text{train}}} [S_g(\pi_{f_\Lambda})]$$

Layer	What is learned	When	Analogy
Outer Λ	Low-level primitive library, modeling method, pretrained model	Offline (G_{pub} + community games); keeps accumulating during evaluation if allowed	Physical fitness, nerves and muscles
Inner h	This game's T_h, G_h, ϕ_h	At evaluation, per game, on the spot	One specific routine

- Private-set mechanics are deliberately out of distribution relative to the public set, so Λ should store low-level primitives rather than concrete rules.
- Contrast: Lux AI S3's inner layer only infers parameters (rules are known); ARC's inner layer must infer the rule structure itself.

4 Kaggle Offline / Preview Methods

No offline method learns an explicit T_h : they either learn a local predictor or keep h implicit in an LLM context; Λ is only a pretrained prior or hand-written heuristics.

4.1 StochasticGoose (Preview 1st, 12.58%)

The inner layer learns only whether the frame will change, not T_h or G_h :

$$q_{\theta}(x, a) \approx P(\text{frame changes} \mid x, a), \quad \pi(a \mid x) \propto q_{\theta}(x, a)$$

- q_{θ} is a CNN trained online; the click head outputs a 64×64 probability map; training data is a buffer of up to 200k entries deduplicated by (x, a) hash; the loss is binary cross-entropy.
- Limitation: “changes” $\neq$ “moves toward G ”; one experiment shows that under current scoring even a perfect q_{θ} gains almost nothing.

4.2 Blind Squirrel (Preview 2nd, 6.71%)

h is an exact lookup model (state graph) plus a value model:

- State graph: nodes are hashed x , edges are (x, a, x') ; self-loops and cycle-inducing actions are pruned.
- After a score increase, states on the level are back-labeled with steps-to-milestone to train a ResNet18 value model $V_{\psi}(x, a)$ that ranks actions.
- Limitation: a lookup table does not generalize; unseen x cannot be predicted.

4.3 Duck (Milestone 1 1st; semi-private 1.21%, public 1.60 ± 0.45)

- Λ : pretrained prior of Qwen 3.6 27B.
- Inner layer: the LLM writes state as variables in a REPL and writes its own analysis code; ϕ is a semantic abstraction and the raw numeric grid is hidden from the Python tool; old messages are evicted, keeping only the system prompt and recent history. h is implicit in context and code, with no explicit L_{fit} .
- Lesson: keep the harness light; hand-crafted tools hurt.

4.4 Reki (Milestone 1 2nd, score not found)

- Λ : Gemma-4-31B plus a hand-written click prior (prefer small, rare-colored, button-like shapes).
- Inner layer: π_{LLM} reads annotated renders of the last few frames each step and outputs a change description, a plan and the next 1–4 actions; reflection memory refreshes about every 10 steps; object classes whose clicks never caused a change go into a dead set and are not clicked again on that level.

4.5 forge (Milestone 1 3rd, score not found)

Same structure as Reki, plus candidate-action generation, a scoring arbiter and confidence hints; its best run had exactly these extras switched off.

4.6 AERA (paper; public 0.2116, private 0.30)

Λ is Qwen2.5-0.5B; the inner pipeline is explore $\rightarrow$ verify $\rightarrow$ plan; details not read.

5 Online Frontier Methods (25 Public Games)

All of these build an explicit executable T_h , and the public set has reached RHAЕ 100. But every result uses frontier API models on the public set with no held-out evaluation; Λ is “a human-written framework plus LLM priors” and is not updated during evaluation.

5.1 Rodionov EWM (GPT-5.5: 58.12, 15 games fully solved; GPT-5.4: 41.29, 8 games)

- h : a Python codebase with fixed interfaces ϕ_h (state representation), T_h (transition prediction), G_h (goal test) and a planner.
- Modeling cost: verifiers run after every edit and require $L_{fit}(h; D_t) = 0$ (exact replay); as evidence grows, refactoring lowers $L_{len}(h)$, which the author describes as a proxy for an MDL simplicity bias.
- Planning: plan in T_h , compare predictions step by step during execution, and stop to revise h at the first mismatch.
- Each playthrough starts from a clean workspace, so the outer layer never accumulates. A follow-up paper compares ewma_sv (executable + simplification + replay verification) with twma (purely textual h).

5.2 Tycho (Opus 5 / GPT-5.6 Sol: 100.00; Opus 4.8: 88.49, no-model baseline 79.07)

- Formalization: parameterized rendered deterministic Moore machine; actionable frames are separated from animation, level-completion and game-over frames.
- h : a simulator generated and revised on the spot for each game; Λ is the fixed prompts, evidence interface, actor–builder protocol, verifier and planner.
- Key variable: when to pay for building a model and when to use it. Among four scheduling policies, “the actor requests the builder” works best; automatically repaired models are more accurate yet lower RHAЕ.
- Cost: about \$2.99k per Opus 5 run.

5.3 OPINE-World (Opus 4.8: 78.40, same-model comparison in the Tycho paper)

- ϕ : parses X_t into an object set $S_t = \{o^i\}$ with $o^i = (\kappa^i, \tau^i, v^i)$.
- Learning: pair objects across S_t and S_{t+1} , compute effect signatures and fill effect-table rows (τ^i, a_t, u^i) ; Dirichlet row diagnostics, type uncertainty and “ontology error” measure whether the abstraction is adequate.
- Only once the ontology is stable does it synthesize an executable T_h , verified by full next-state equality.

5.4 Kepler (Opus 5: 100.00)

“Auditable world models”; about \$778 per Opus 5 run (at September 1, 2026 API list prices), with 36% fewer tokens than Tycho. Method details not read.

5.5 Symbolica Arcgentica (36.08 on launch day)

Recursive agent orchestration that interacts with the game through code; h is implicit, with no explicit modeling cost.

5.6 ARA (AgentNativeResearchLab, score not found)

Builds a structured h during play and consults it when cold exploration fails; keeps a prediction ledger with confidences (each query of h and whether it later proved right).

6 Other Methods and Background

A few ARC-AGI-3 approaches have no formal score; background work is grouped by whether it acts on the inner or outer layer. Most citations are from memory and marked “memory”.

6.1 Other ARC-AGI-3 Approaches

Method	Layer	In this paper’s notation	Note
Graph-Based Exploration (Rudakov, Shock, Cowley)	Inner	h = state graph, same line as 4.2	arXiv 2512.24156
Game-classifying open-source solver	Outer	Λ = hand-written game classes + per-class solvers; ϕ by connected components	URL not recorded
Local LLM writes a Python world model (design only)	Inner	Local LLM generates T_h , verifies on D_t , plans in the model, reuses across levels	Design only, no score
Self-written simulator harness	Inner	Once rules stabilize, the agent writes sim.py as T_h to predict without real actions	No score recorded

6.2 Background Methods

Method	Layer	What it learns	Relation to this problem	Source
EMPA (Tsividis et al.)	Inner	Bayesian inference over VGDL rule space H + exploration + planning	Inspiration for ARC’s efficiency scoring; VGDL only expresses simple collision interactions	Cited by ARC
TheoryCoder	Inner	LLM synthesizes code for T_h for bilevel planning	Goes beyond VGDL’s expressiveness	TMLR 2025
WorldCoder (Tang, Key, Ellis)	Inner	Builds T_h by writing code and interacting	Early programmatic world model	2024
Code World Models + MCTS (Dainese et al.)	Inner	LLM generates code T_h guided by MCTS	Same line	2024
PoE-World	Inner	T_h = product of programmatic experts	Compositional rule representation	2025
DreamCoder	Outer	Learns a reusable program primitive library Λ from many tasks	Closest paradigm to “primitive library as prior”	memory
Voyager	Outer	Continually growing agent skill library	Cross-task skill reuse	memory
Dreamer / MuZero	Inner	Neural T_h + planning / search in imagination	Needs heavy interaction; unsuited to few-shot first contact	memory
MAML / RL ²	Outer	ω = initialization / weights of an adapting RNN	Standard meta-learning forms (Section 3.4)	memory

7 Comparison Table

The higher the score, the closer the inner layer is to “explicit $T_h + L_{fit}$ verification + planning in the model”; no method learns an explicit outer layer Λ .

Method	Setting	Score	Outer Λ	Inner h	Modeling cost	Play cost
Tycho	Online, Opus 5 / GPT-5.6 Sol	Public 100.00	Fixed framework + LLM	Executable T_h	L_{fit} verification	Plan in model; actor decides when to build/use
Kepler	Online, Opus 5	Public 100.00	Auditable WM framework + LLM	Not read	Not read	Not read
Tycho	Online, Opus 4.8	Public 88.49	Same	Same	Same	Same
OPINE-World	Online, Opus 4.8	Public 78.40	Object parsing + effect table	T_h synthesized after ontology stabilizes	Dirichlet diagnostics + ontology error; next-state equality	Not read
Rodionov EWM	Online, GPT-5.5	Public 58.12	Fixed interfaces + verifier + refactoring	Python ϕ_h, T_h, G_h	$L_{fit} = 0$ + lower L_{len} (MDL)	Plan in model, compare on execution
Symbolica	Online, frontier model	Public 36.08	LLM	Implicit	None explicit	None explicit
StochasticGoose	Preview	12.58%	None	$q_\theta(x, a)$	Binary cross-entropy	$\pi \propto q_\theta$
Blind Squirrel	Preview	6.71%	None	State graph + V_ψ	None (exact record)	Value regression
Duck	Kaggle offline	Semi-private 1.21%	Qwen 3.6 27B	Implicit in context / code	None explicit	None explicit
AERA	Paper	Private 0.30	Qwen2.5-0.5B	Explore $\rightarrow$ verify $\rightarrow$ plan	Not read	Not read
Reki	Kaggle offline	Not found	Gemma-4-31B + click prior	Reflection memory + dead set	None explicit	1–4-step plan
forge	Kaggle offline	Not found	Same as Reki	Same as Reki	None explicit	None explicit
ARA	Online, Opus 4.8 etc.	Not found	LLM	Structured h	Prediction ledger	Not read

Note: online scores are public-set RHAЕ (out of 100); offline and Preview scores use different protocols and are not directly comparable.

8 Gaps and Open Questions

Two gaps: an explicit outer layer Λ (a primitive library learned offline and updatable during evaluation), and an explicit T_h under offline local-model conditions. “How to learn” (which action to explore, how to weigh hypotheses) is still decided implicitly by the LLM.

- Milestone 1 2nd and 3rd place scores; Milestone 2 (Sept 30 deadline) results
- Method details and cost definitions of Kepler, AERA and ARA
- Whether Kaggle runs the 110 games sequentially in one process; whether rules allow accumulating Λ across games during evaluation

- Whether rules allow training local models on data generated by API models
- Whether RESET counts toward $n_{g,\ell}$

Sources

- Executable World Models for ARC-AGI-3 (arXiv 2605.05138)
- Do Coding Agents Need Executable World Models... (arXiv 2607.15439)
- Tycho (arXiv 2607.28287) · code
- Kepler (arXiv 2610.00834)
- OPINE-World (arXiv 2607.01531)
- Rodionov code and run artifacts
- Duck harness
- ARA trajectory example
- Awesome Code World Model